

RustQuests Admin Guide

Sep 27, 2026 · @Paul Butler

What RustQuests is

RustQuests adds a story-driven quest line to a vanilla Rust server: four named trader NPCs in hidden train-caboose shops, a chain of quests that runs through all four, and a finale vault that needs every chain done. Everything is data-driven from JSON under `oxide/data/RustQuests/`, so text, rewards and trader looks can be tuned live without a redeploy.

The story is not fixed. Each wipe rolls a **wipe bible** from a pool of six story templates and remixes the one it picks: chain order, which trader holds which secret, quest variants, timed story beats, and one late-wipe choice with real consequences. The same server tells a different story every wipe, and the traders remember earlier wipes.

Two optional companions plug in: SLM chat (the traders answer free-text questions in character) and trader voice (every line is spoken through an in-game radio). Neither is required for the core game.

Install and first boot

Two files matter: `RustQuests.cs` goes in `oxide/plugins/`, and the pamphlet JPGs go in `oxide/data/RustQuests/pamphlets/` (the release zip's `data` folder drops straight into `oxide/`). Without the JPGs everything still runs, but The Unpaid's pamphlets never appear in crates.

On first load the plugin writes its config and every content file, then rolls the wipe's story. About 60 seconds after boot it places the world by itself:

1. Four cabooses, one per trader, at a random flat spot in her biome (jungle, temperate, arid, arctic), off monuments, with lights on and a map marker.
2. One trader in each cashier booth, with her fixed face and outfit.
3. Five codelocked stashes hidden for the finale chain, plus The Unpaid's field sheet and dead-drop box in a ring outside a bound monument.

A healthy boot log reads, in order: `Loaded plugin Rust Quests`, a `season` line (fresh roll or carried), four `Shop prefab ... placed` lines, four `Trader ... spawned` lines, and no `chain is broken` warnings. Run `rq.status` in the console to see the same picture at any time.

Permissions: server console and auth level 2 can run every `rq.*` command. Grant `rustquests.admin` to anyone else who needs them; the permission also exempts the holder from the 25 m no-build zone around each shop.

The four traders and the chain

The traders are a chain, not a menu. Sonia is always the head: the welcome note every player gets on first wake points at her railcar. Each trader vouches for you to the next, and a trader you have not been introduced to withholds her name and offers no work. The vault finale opens back at Sonia and needs all four personal finales claimed.

Trader	Biome	Trade	Chain theme
Sonia	Jungle (temperate fallback)	Hunting	Skins, meat, a knife, the jungle hunt
Rebecca	Temperate	Farming and electrics	Soil, water, power, automation
Alexa	Arid	Vehicles and benches	Parts, engines, a garage
Olivia	Arctic	Munitions	Wood, cordite, a Bradley recall

The classic order is Sonia, Rebecca, Alexa, Olivia, then back to Sonia. Most story templates shuffle the three behind Sonia, so the order on your server is whatever `rq.bible` prints for this wipe. The dialog handles any order: a trader's closing line names the next one, gives a locator hint and a map grid, and the last trader sends the player back to Sonia for the vault.

Each trader has a fixed face, an authored outfit from Workshop skins, and her own register: Sonia is direct, Olivia cold, Alexa sarcastic, Rebecca motherly. All of it lives in `traders.json` and can be recast there.

How a quest works

A player presses E on a trader, reads her offer, accepts, does the work, and returns to claim. Progress is per player and per wipe, stored in `players/<steamid>.json`. Any number of players can run the same wipe's chains in parallel.

Every quest is a record in `quests.json` with an id, a trader, a prerequisite quest, objectives, rewards, and its dialog. A quest moves through four states: offered, active, complete (objectives done, not yet claimed), rewarded. Some quests carry an accept cost (scrap paid up front) or on-accept grants (tools she hands you for the job).

Objective type	What the player does	Checked by
turnin	Bring N of an item to the counter	Inventory at claim
kill	Kill N of a target (scientists, the Bradley)	Kill hooks
visit	Reach a place	Position sweep
readnote	Read a specific note	Note pickup
code	Open a codelocked stash	Lock hook
drop	Deposit items in The Unpaid's dead-drop box	Box contents, auto-claims
plant	Leave a pamphlet at N distinct monuments	Placement ledger

Rewards are vanilla scrap and items paid to inventory, with a drop-at-feet fallback when it is full. Dialog uses tokens such as `{rewards}`, `{count}` and `{target}`, so retuning a number in the JSON updates what she says.

The `/quests` journal has two tabs. The Work shows chain progression and where to go next. The Story keeps the finished quests readable in this wipe's voices, including the ending the player actually earned.

How storylines work

At the first boot of a season the plugin rolls a **wipe bible**: one seeded record in `state.json` that fixes this wipe's story. Everything else reads from it. The seed makes the roll reproducible, so a reroll with the same seed lands the same monuments and picks.

The roll picks one template from `templates.json` (weighted, and it prefers not to repeat last wipe's), then remixes inside it:

- **Chain order.** Sonia stays first; the other three shuffle. Quest prerequisites and unlock wiring are resolved from the bible at read time, so the JSON is never rewritten.
- **Roles.** The template deals a secret role to each trader (the press, the witness, the clerk, and so on). A role overlay from `overlays.json` recolors her scripted dialog and her SLM persona while she carries it.
- **Secrets.** Facts such as who prints the pamphlets or who had a seat on the manifest are rolled once. Traders keep them by omission: nobody is told the answer, and authored slip lines only surface at high warmth, as checkable tells.
- **Variant slots and pools.** Some quests roll one of several variants for the wipe, or a different turn-in item and count. Variants share the classic quest's id, so the chain can never break.
- **Beats, the verdict, and consequences.** Covered in the next section.

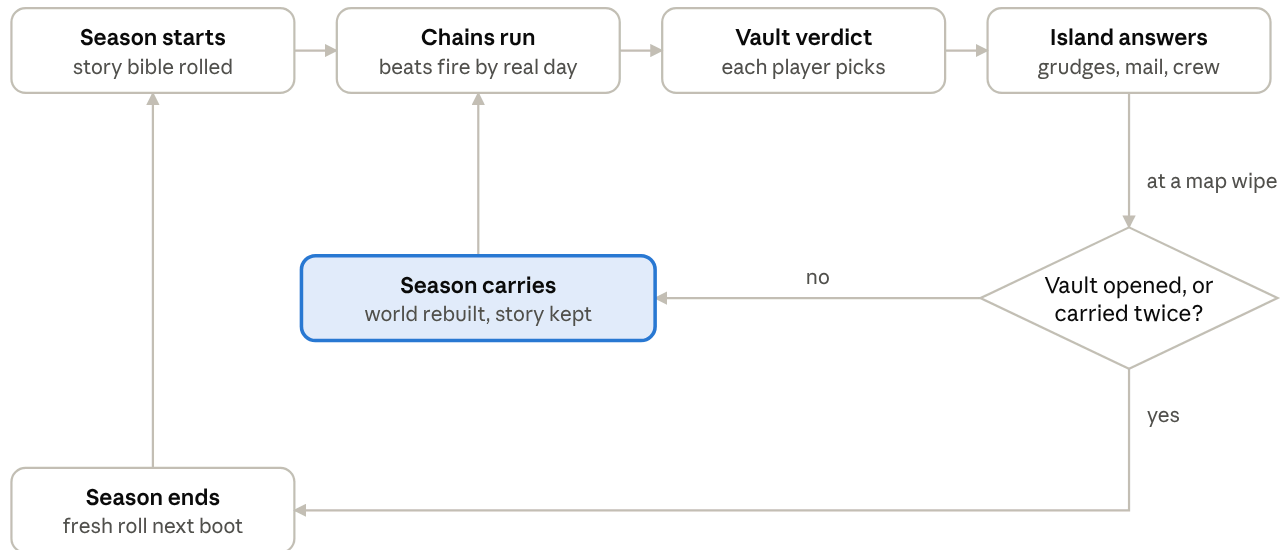
Template	Premise	What is rolled
Four Keys	The classic four chains in fixed order	Nothing beyond slots and pools
The Press	One trader secretly prints the pamphlets	Chain, roles, press identity
The Manifest	40 seats for 200 people; who had one?	Chain, roles, the seated, the leak
Yours, C.	Who signs the typed welcome note?	Roles, the author (three possible answers)
The Recall	Spent company casings; someone opened the magazine	Roles, the digger (never scripted, deduction only)
The Count	The wipe tally goes public	Roles, the tallier; quotes real earlier wipes

A Four Keys wipe plays exactly like the original story. Any other template keeps the same quests and rewards but changes who knows what, in what order you meet them, and what the vault verdict is about. `rq.bible` prints the whole roll with secret values hidden; `rq.bible secrets` reveals them.

Mid-wipe story machinery

The story keeps moving after the roll. Four mechanisms fire during a wipe, all driven by real calendar days and by what players actually do.

A season runs until the vault opens or two map wipes pass



life of a season · roll, chains, verdict, consequences, wipe

The loop back from Season carries means a map wipe rebuilds the world but keeps the bible and every player's progress; the loop from Season ends starts a new roll.

Beats. A template can schedule story arcs to go live N real days into the wipe, optionally gated on any player having claimed a named quest. Firing is additive: a beat activates an arc, places a note or a print, and never removes anything. A boot catches up on past-due beats in order. The Unpaid's FIELD NOTES reprint and The Manifest's PAGE SIX are beats.

The verdict. One quest per template is the choice point, normally the vault. Its claim dialog becomes a picker with two or three options and no default. Each player's pick is recorded on their own record and rewrites their ending in the journal; the island's first pick sets the world flavor for everyone (a story arc goes live and a canon line joins every SLM prompt).

Grudges. Consequences are computed from a player's picks whenever a trader talks to them, never stored. Four severities: reserved (a cooler line), frozen (transactions only, she never explains), grieving (one line, forever), and barred (she evicts the player, refuses chat, and her caboose's slot machines and card tables refuse them too).

Delayed pieces. Some picks send paper and supplies a rolled number of real days later, delivered the next time the player is awake past the due day, telegraphed by one narration line. House rule: aggression arrives unsigned, kindness signs.

The collections crew. One verdict (exposing the press) earns a night visit: on the first night after day 2 that the picker is online, awake and outside a safe zone, three unsigned guns in company hazmats walk in from 35 to 55 m out. They deal half damage, never enter building privilege, leave after 8 minutes, drop scrap, and come once per wipe. The next pamphlet denies they exist.

The Unpaid

The Unpaid are ex-Cobalt ground staff who were never paid out and may not exist. They are the second quest giver, and they have no face: their quests arrive on paper, auto-claim when the work is done, and chain into the next sheet without a counter to return to.

- **Pamphlets in crates.** Each spawning `crate*` container has a `PamphletLootChance` (default 5%) of carrying one of their prints, an inspectable photo item stamped from the JPGs in the pamphlets folder. Some prints are arc-gated in `pamphlets_meta.json` and only circulate once a beat or verdict has unlocked them; `rq.pamphlet.status` shows each print's current verdict.
- **Paper that grants work.** Picking up a print grants A READER. Reading the field sheet placed near the bound monument completes it and starts BACK PAY, three dead drops in a row.
- **Dead drops.** A `drop` objective is satisfied by depositing the asked items in The Unpaid's box; the plugin credits it, auto-claims, and pays scrap without a trader visit.
- **Planting.** A `plant` objective asks the player to leave prints at distinct monuments; a ledger stops the same monument counting twice.
- **They count the wipes.** Their sheets quote real history through tokens: `{wipes}` (the season number), `{lastwipe.template}` and `{lastwipe.choice}` (what happened last season), and `{fieldnote}` (a FIELD NOTES number claimed once per wipe, never reused). On a first wipe these fall back to safe wording.

Their register is capitals and short sentences; C., the welcome note's author, writes lowercase and warm. Keep to that if you author more of their paper in `notes.json`.

Wipes and seasons

A map wipe does not end the story by itself. The plugin detects a new map by its map key at boot, then decides between two paths:

Condition at boot	What happens
Vault (<code>grand_finale</code>) never claimed and carried fewer than <code>SeasonCarryOverMaxWipes</code> times (default 2)	Season carries. The bible and every player's progress stand. The world is rebuilt for the new map: shops, stashes, notes and drops re-placed, monument bindings re-surveyed. Log: <code>season CARRIED (n/2)</code> .
Vault claimed by anyone, or already carried the maximum, or the cap is 0	Season ends. State resets, a fresh bible rolls, and last season's template and first verdict are distilled into cross-wipe memory. Log: <code>season ends (...)</code> .

`rq.bible` and `rq.status` print what the next boot will do before it happens. To end a season by hand, `rq.wipe.reset` . To rehearse a carry without a wipe, `rq.wipe.carry` rebuilds the world and leaves the story and progress alone.

Set `SeasonCarryOverMaxWipes` to 0 in the config to get the older behavior, one story per map.

What every wipe does regardless of path: the wipe counter `{wipes}` ticks, four shops self-place in their biomes within a minute, five finale stashes are re-hidden with new codes, and each player gets the welcome note again on first wake. Per-player progress files are never touched by a carry and are wiped only at a season end.

After any wipe, read the boot log for the `season` line and the four shop and trader placements, then run `rq.bible` and walk one trader's chain start to finish before players arrive.

Optional companions: chat and voice

Both are off by default and invisible until configured. Every quest, verdict and consequence works fully scripted without them.

SLM trader chat. Set `SLMEndpoint` (and `SLMModel`, plus `SLMApiKey` for hosted) in `oxide/config/RustQuests.json` to any OpenAI-compatible chat endpoint, such as a local Ollama or OpenRouter. Each unlocked trader then gains a Chat button in her dialog. What the model sees is composed per call: her persona and role overlay, the wipe's canon facts, the player's current quest and objective progress, her warmth tier for that player, and up to four remembered sentences from earlier sessions. Stash codes and grids are never in the prompt. Sessions run 5 minutes with a 60-minute cooldown per player and trader, an hourly call budget (default 60), and a circuit breaker that pauses chat for 5 minutes after three failures. Warmth grows as the player clears her chain and unlocks the authored slip lines about the wipe's secrets.

Trader voice. Two extra files: the `RustQuestsVoice.cs` plugin in `oxide/plugins/` and the `Oxide.Ext.RustQuestsVoice.dll` extension in `RustDedicated_Data/Managed/`, which needs a server restart to load. Point `TTSEndpoint` in `oxide/config/RustQuestsVoice.json` at a Kokoro-FastAPI container on a local GPU, or at OpenRouter with an API key and `TTSFormat: "pcm"`. Every dialog line and chat reply is then rendered to Ogg, cached on disk by content hash, and played through a hidden radio behind her counter on completely vanilla clients. Each trader has her own voice id in that config. A TTS failure never blocks the text.

Check either with `rq.slm` and `rq.slm.test <trader> <message>`, or `rq.voice.status` and `rq.voice.test <trader>`.

Data files and live tuning

All content is JSON under `oxide/data/RustQuests/` and hand-editable. Numbers in `quests.json` take effect on plugin reload; trader and overlay edits apply with `rq.trader.reload`, no full reload needed.

File	Holds	Edit it to
<code>traders.json</code>	Identity, outfit skins, shop, booth offsets, dialog, persona, warmth tiers	Recast a trader, retune her lines, move her in the booth
<code>quests.json</code>	Every quest: objectives, rewards, accept cost, dialog, prerequisites	Retune rewards and counts

File	Holds	Edit it to
<code>templates.json</code>	The six storylines: chains, roles, secrets, slots, beats, verdicts, consequences	Reweight or disable a template, adjust beat days
<code>overlays.json</code>	Per trader-and-role dialog recolors	Change how a role sounds
<code>notes.json</code>	Every piece of paper, with tokens	Reword lore
<code>stashes.json</code>	Stash loot tables by tag	Retune finale loot
<code>pamphlets_meta.json</code>	Per-print gates and grants	Control which prints circulate when
<code>state.json</code>	Wipe id, the rolled bible, placements, cross-wipe memory	Read only; never hand-edit on a live server
<code>players/<steamid>.json</code>	Per-player progress, choices, memories	Delete one to reset a player, or use <code>rq.quest.reset</code>

Shipped updates merge add-only. A new plugin version adds quests, notes and templates it has never seen and fills blank fields, but never overwrites a value you set. That protects your tuning, and it also means a wording or reward change in a new release does not reach a running server on its own. The sync commands exist for when you want shipped values to win:

- `rq.quest.synctext [quest-id]` pushes wording only and leaves tuned rewards alone.
- `rq.quest.syncrewards <trader|quest-id>` pushes objectives and rewards and **discards** your tuning for what it touches, so scope it to one trader.
- `rq.trader.synctext [trader]`, `rq.note.synctext [tag]`, `rq.overlay.synctext`, `rq.stash.syncloot [tag]`, `rq.template.sync [id]` follow the same pattern.

Config keys worth knowing in `oxide/config/RustQuests.json`, beyond the shop placement and SLM keys above: `ShopNoBuildRadiusMetres` (25 m, 0 disables), `ProtectWorldObjects` and `TraderGodMode` (both on), `WelcomeNote`, `PamphletLootChance`, `SeasonCarryOverMaxWipes`. Two optional progression hooks are off by default: `TechTreeLockWorkbench1..3` and `TechTreeLockEngineering` block scrap

unlocks in that tech tree so blueprints must be researched or found, and `WorkbenchGateWorkbench1..3` and `WorkbenchGateEngineering` let a player craft or place that workbench only after finishing a named trader's chain.

Admin command reference

All commands run from the server console, or in-game by auth level 2 and `rustquests.admin` holders. The `rq.ui.*` commands are internal dialog plumbing and are not listed. The full table with every argument is in the README.

Purpose	Commands
See what is going on	<code>rq.status</code> , <code>rq.bible [secrets]</code> , <code>rq.quest.list</code> , <code>rq.pamphlet.status</code> , <code>rq.piece.status [player]</code> , <code>rq.slm</code>
Fix a player	<code>rq.quest.reset <player></code> , <code>rq.unlock <player></code> <code><trader></code> , <code>rq.piece.deliver [player]</code>
Fix a trader or shop	<code>rq.trader.reload</code> , <code>rq.trader.spawn /</code> <code>rq.trader.remove / rq.trader.tp</code> , <code>rq.shop.offset</code> , <code>rq.trader.move</code> , <code>rq.trader.nudge <m></code> , <code>rq.trader.face [n]</code> , <code>rq.shop.autoplace / rq.shop.remove</code> , <code>rq.shop.lights</code> , <code>rq.shop.diagnose</code> , <code>rq.shop.biomescan</code>
Push shipped content	<code>rq.quest.synctext</code> , <code>rq.quest.syncrewards</code> , <code>rq.trader.synctext</code> , <code>rq.note.synctext</code> , <code>rq.overlay.synctext</code> , <code>rq.stash.syncloot</code> , <code>rq.template.sync</code>
Notes, stashes, prints	<code>rq.note.place <tag> / rq.note.remove <tag></code> , <code>rq.note.welcome</code> , <code>rq.stash.place <tag></code> <code><code> / rq.stash.remove / rq.stash.reroll</code> , <code>rq.pamphlet.give</code>
Drive the story (dev)	<code>rq.bible.reroll [template]</code> , <code>rq.beat.fire</code> <code><id></code> , <code>rq.crew.visit [player] / rq.crew.end</code> , <code>rq.wipe.carry</code> , <code>rq.wipe.reset</code> , <code>rq.tree <jungle</code>

Purpose	Commands
Chat and voice	<code>rq.slm.test <trader> <msg></code> , <code>rq.slm.prompt <trader> [tier]</code> , <code>rq.voice.status</code> , <code>rq.voice.test <trader> [text]</code> , <code>rq.voice.rebuild</code>

The dev row changes the live story. `rq.bible.reroll` orphans any in-flight arc progress and `rq.wipe.reset` ends the season, so use them on a test boot or with intent.

Troubleshooting

Start with the log. Oxide writes `oxide/logs/oxide_<date>.txt` and the plugin keeps its own under `oxide/logs/RustQuests/`. Then `rq.status` for the world and `rq.bible` for the story.

Symptom	Likely cause	Fix
Traders stand under the cabooses	<code>traders.json</code> has all-zero booth offsets (installs before 2.20.4)	Update to 2.20.4 and <code>rq.trader.reload</code> ; or stand in the booth and run <code>rq.shop.offset</code> per trader
A shop never places	No flat spot found in her biome; log shows placement retries	<code>rq.shop.diagnose</code> , <code>rq.shop.biomescan</code> ; raise <code>ShopFlatnessMetres</code> or turn off <code>ShopBiomeStrict</code> ; <code>rq.shop.autoplace</code> to retry now
Caboose lights off	Player flipped a switch; the lights are a two-way circuit	<code>rq.shop.lights</code> , or the healer fixes parity within a minute
No pamphlets ever drop	No JPGs in the pamphlets folder, or <code>PamphletLootChance</code> is 0	Copy the prints in; log should say <code>Pamphlet press loaded: N print(s)</code>
A player is stuck on a chain	Prerequisite not met, or a trader she needs is not yet unlocked	<code>rq.status</code> shows per-trader unlock wiring; <code>rq.unlock <player> <trader></code> to vouch them through

Symptom	Likely cause	Fix
Chat button missing	<code>SLMEndpoint</code> empty, or the trader is not yet unlocked for that player	Configure the endpoint; check <code>rq.slm</code>
Chat stops answering	Circuit breaker tripped after three failures, or hourly budget spent	<code>rq.slm</code> shows breaker and budget; <code>rq.slm.test</code> round-trips the endpoint; cold model loads can take 15 s, so keep <code>SLMTimeoutSeconds</code> at 25
Voice silent, text fine	Extension not loaded (needs a restart), endpoint down, or breaker tripped	<code>rq.voice.status</code> ; <code>rq.voice.test <trader></code>
A new release's wording did not change	Merge is add-only by design	Scoped sync command for that content
Story looks wrong after a wipe	Season carried when you expected a reset, or the reverse	Read the <code>season</code> boot line; <code>rq.wipe.reset</code> ends a season by hand
Chain warning in the log	An active quest's prerequisite is not active this wipe (usually after a hand edit)	Fix the quest or template edit, or <code>rq.template.sync</code> to restore shipped wiring

When reporting a bug, include the plugin version from the load line, the `season` line, the output of `rq.status` and `rq.bible`, and the relevant `traders.json` or `quests.json` if you have edited them.